SudokuScan: From a Photograph of a Printed Puzzle to a Digital 9×9 Grid

Kerem Ataç, 042201043
COMP 482 Computer Vision, Project 1

***Abstract*—This report describes a pipeline that reads a printed sudoku puzzle from a phone photograph. The four outer corners of the grid are found with the Harris detector, the puzzle is rectified with two affine models and a homography, and the printed digits are read with Tesseract. On the 20 development images the mean corner error is 0.28 % of the puzzle side and all cells are correct. On my 10 photographs the corner error is 0.27 %, cell accuracy is 0.993 and 8 of the 10 grids are read without any error. The comparison of the three models shows that an affine map is only acceptable for near-frontal views. The four bonus parts (solver, solution drawn on the photograph, many-point homography and orientation detection) are described in Section VIII.**

I. Introduction

A sheet of paper is a plane, so its image under a pinhole camera is related to the flat puzzle by a homography. The task is to undo this mapping automatically and read the 81 cells. The difficulties are (i) finding the four outer corners among thousands of corner-like points produced by grid intersections, digits and surrounding text, (ii) uneven lighting and (iii) an OCR engine that is designed for lines of text, not single digits.

The pipeline has six stages, each a separate function in `sudoku_scan.py`: preprocessing, Harris corner detection with automatic corner selection, rectification with three models (A3, A4, H4), their comparison, cell extraction with digit recognition, and a validity check (Table I). It uses only OpenCV, NumPy and Tesseract.

The personal dataset has 10 photographs of 3 printed puzzles (KrazyDad, Web Sudoku and a newspaper puzzle booklet), covering C1 twice, C2 five times, C3 twice, C4 four times and C5 three times. The puzzle side is between 1330 and 2030 px. Corners were annotated by clicking on a zoomed view of each photograph; the grids were typed once per puzzle.

TABLE I
Pipeline overview

Stage	Function	Method
1	`preprocess`	downscale to 1000 px, grey, 3×3
	`binarise`	Gaussian blur, ratio threshold against a closing
2	`harris_candidates`	`cornerHarris`, relative threshold, non-maximum suppression
2	`find_region`	component with about 81 holes,
	`select_corners`	largest quadrilateral of candidates, `cornerSubPix`
3	`rectify`	A3, A4 and H4 to 450×450, corner residuals
4	`compare_models.py`	residuals, line deviation and cell accuracy against obliqueness
5	`find_digits`	empty-cell rule on connected
	`prepare_digit ocr`	components, Tesseract `--psm 10`
6	`find_conflicts`	repeated digits in a row, column or box

II. Corner Detection

A. Preprocessing

The image is downscaled so that its longest side is 1000 px and converted to grey. All later sizes are fractions of this working size, so a 1000 px and a 4000 px photograph are treated identically; corners are scaled back to original pixels at the end.

Lighting is normalised by binarisation. After a 3×3 Gaussian blur, the local paper brightness P is estimated with a morphological closing (kernel 2 % of the image width), which removes dark structures thinner than the kernel, such as lines and digits. A pixel I is ink when $I < 0.85\,P$. Because the test is a ratio, it gives the same result in a bright and a dark photograph, which a fixed offset from the local mean (`cv2.adaptiveThreshold`) does not.

B. Harris response and candidates

`cv2.cornerHarris` is applied to the binary image. The response is $R = \det M - k\,(\operatorname{trace} M)^2$, where M is the structure tensor summed over a `blockSize` neighbourhood of gradients computed with a Sobel filter of aperture `ksize`. Working on the binary image gives every ink edge the same contrast, so a threshold relative to the maximum response does not discard corners that lie in shadow.

Candidates are pixels with $R > 0.05\,R_{\max}$ that are also local maxima: a pixel is kept when it equals the grey-scale dilation of R with a window of 0.5 % of the image width. The window must be smaller than the thickness of the outer border; with a 1 % window the outer and inner corner of the thick border merged into one peak, and the corner error of several images rose above 1 %.

C. Corner selection rule

Region. In the binary image the sudoku grid is one connected component with 81 holes. For every component found by `cv2.findContours` (`RETR_CCOMP`) the holes with an area between $1/400$ and $1/5$ of the component area are counted. Components that touch the image border are discarded, since the puzzle is fully visible. The puzzle is the component whose count is closest to 81. Text and the page

TABLE II
HARRIS PARAMETER STUDY (PERSONAL DATASET, 10 IMAGES)

Parameter	Value	Candidates	Error (%)	Failures
blockSize	2	2380	0.33	0
	3	2458	0.27	0
	5	2033	0.27	0
	7	1994	0.24	0
	9	1982	0.24	0
	13	2458	0.34	0
ksize	**3**	2033	0.27	0
	5	1578	0.27	0
	7	1537	0.27	0
k	0.02	2731	0.27	0
	0.04	2033	0.27	0
	0.06	1863	0.27	0
	0.10	1647	0.46	0
	0.15	1533	0.51	0
	0.20	1401	1.63	1
threshold	0.01	3510	0.27	0
	0.05	2033	0.27	0
	0.10	1695	0.27	0
	0.20	1439	0.51	0
	0.40	879	8.04	7
	0.60	225	23.77	10

edge have almost no such holes, an empty rectangle has one hole that is too large, and a crossword has too many.

Corners. Only Harris candidates that lie on this component (dilated by one pixel) are kept, which removes text printed next to the grid. The four outer corners are the four points of the convex hull of these candidates that span the quadrilateral of largest area. They are ordered TL, TR, BR, BL by their angle around the centroid and refined with `cv2.cornerSubPix`. Contours are used only to find the region; all coordinates come from the Harris detector.

D. Parameter study

One parameter was varied at a time around the chosen values (`blockSize` = 5, `ksize` = 3, k = 0.04, threshold = 0.05) on all 10 personal photographs (Table II, Fig. 1). A failure is an image with no puzzle found or a corner error above 5 %.

The corner error is flat over a wide range, because the selection rule only needs the four outer corners to be among the candidates. A small `blockSize` (2) gives a noisy response and a large one (13) blurs it and shifts the peak away from the true corner. Sizes 7 and 9 are 0.03 % better than 5 on these photographs, which is below the precision of the hand-clicked ground truth, and on the development set 9 is slightly worse than 5, so 5 was kept. `ksize` changes the number of candidates but not the error. A larger k penalises edge-like structure more strongly; on real photographs the error already rises at $k = 0.1$ and one image fails at $k = 0.2$. The threshold is the most sensitive parameter: from 0.2 the outer corners themselves start to be rejected, and at 0.4 seven of ten images fail. The chosen values lie inside the flat range of every parameter, and the development set (20 images) shows the same pattern with a flat error of 0.28 %.

TABLE III
WHERE A3 SENDS THE UNUSED CORNER BR, IN PIXELS (IDEAL: 450, 450), BY OBLIQUENESS

Obl.	Lands at	Dist.	Obl.	Lands at	Dist.
1.02	(443, 461)	12.8	1.20	(555, 532)	133.5
1.03	(463, 453)	13.4	1.29	(610, 350)	188.4
1.06	(476, 447)	25.8	1.41	(635, 443)	185.0
1.11	(499, 425)	55.1	1.43	(642, 449)	192.3
1.12	(402, 451)	48.2	1.68	(742, 411)	294.9

E. Cases the rule fails on

The rule assumes that the grid is one connected component with closed cells, that nothing else is attached to it and that it does not touch the image border. On the 10 final photographs it never selected a wrong corner. The first version of the rotated photograph did fail: one corner of the grid was outside the frame, the grid component was discarded because it touched the border, and a piece of the title text was selected instead (Section VII). That photograph was retaken.

III. RECTIFICATION

The four corners are mapped to a 450×450 square (50 px per cell) with three models.

A3 (`cv2.getAffineTransform`) is the affine map through TL, TR and BL. An affine map $\mathbf{x}' = A\mathbf{x} + \mathbf{t}$ has six parameters and every point pair gives two equations, so three pairs determine it exactly. The fourth corner is not used; where it lands measures how far the view is from affine.

A4 (`cv2.estimateAffine2D`) is the least-squares affine fit to all four corners. With the reprojection threshold set to the image size all four points are inliers, so the result equals the ordinary least-squares solution. It spreads the error over the four corners; the RMS residual is reported.

H4 (`cv2.getPerspectiveTransform`) is the homography through all four corners. A homography is a 3×3 matrix defined up to scale, so it has eight degrees of freedom and needs four point pairs. The two additional parameters form the last row and make the denominator depend on the position, which models perspective foreshortening.

An affine map preserves parallelism: it turns a square into a parallelogram and can model rotation, scale and shear but no vanishing point. A homography maps lines to lines without preserving parallelism, so it can represent any view of a plane. On the personal dataset H4 maps the four corners onto the target square with a maximum error below 10^{-4} px, while the A3 fourth-corner error ranges from 13 to 295 px and the A4 RMS residual from 3 to 56 px. Table III lists, for each photograph, where A3 sends the corner it does not use; the positions are computed from the A3 matrix and the corners stored in the result files.

IV. AFFINE VERSUS HOMOGRAPHY

Obliqueness. Obliqueness is defined as the largest ratio between the lengths of opposite sides of the detected quadrilateral. It is exactly 1 for every parallelogram, so it ignores

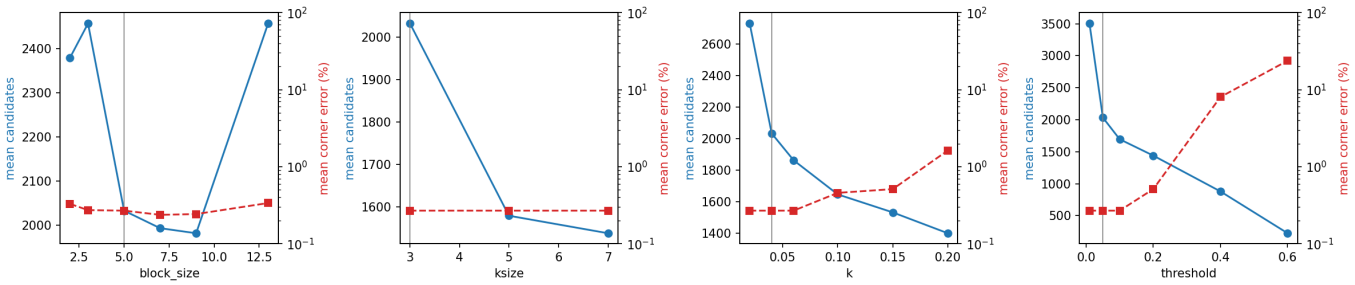


Fig. 1. Number of candidates (blue) and corner error (red, log scale) for each Harris parameter on the personal dataset. The grey line is the chosen value.

TABLE IV
AFFINE VERSUS HOMOGRAPHY BY OBLIQUENESS (PERSONAL DATASET)

Obliqueness	1.02–1.06	1.11–1.20	1.29–1.68
Images	3	3	4
A3 error (px)	13–26	48–134	185–295
A4 RMS (px)	3–6	13–27	38–56
Line dev. A3 (px)	2.7–7.7	8.6–12.1	11.9–14.7
Line dev. A4 (px)	1.3–5.1	5.5–15.6	11.2–17.8
Line dev. H4 (px)	0.4–3.1	0.6–1.8	1.0–10.3
Cell acc. A3	0.91–1.00	0.85–0.86	0.52–0.63
Cell acc. A4	0.98–1.00	0.77–1.00	0.43–0.63
Cell acc. H4	0.99–1.00	1.00	0.94–1.00

in-plane rotation and measures only the perspective part of the distortion. The rotated photographs confirm this: they are rotated by 17° and 21° but have an obliqueness of only 1.11 and 1.02.

Measures. (1) The A3 fourth-corner error and the A4 RMS residual. (2) Grid line deviation: the rectified image is binarised and projected onto each axis; near each of the 20 ideal line positions (every 50 px) the strongest peak of the profile is located, and the mean distance to the ideal position is reported. (3) Cell accuracy of the same recognition stage on each rectification.

Results. Fig. 2 shows the three measures for the 10 personal photographs and Table IV summarises them in three groups.

Both residuals grow steadily with obliqueness. The A3 error is about four to five times the A4 residual, because A3 puts the whole misfit on one corner while A4 spreads it over four. With the affine models the grid lines are several pixels off already at an obliqueness of 1.06, and from 1.2 the deviation saturates at 11–18 px, which means that the profile peaks no longer correspond to the right lines. With H4 the lines stay within about 2 px for seven photographs. The three exceptions (3.1, 4.0 and 10.3 px) are all photographs of the newspaper booklet, whose pages do not lie flat; a curved page is not a plane and no 3×3 transformation can flatten it.

Cell accuracy follows the same order. At 1.02–1.03 all three models read the grid equally well. At 1.06 A3 already loses seven cells and A4 two. At 1.11–1.12 A4 still loses at most two cells while A3 drops to 0.85. At 1.20 A4 falls to 0.77, and above 1.29 both affine models are between 0.43 and 0.63 while H4 remains at 1.00 on every flat page. The single H4 error at 1.02 is an OCR confusion, not a geometric one.

Conclusion. An affine model is an acceptable approximation only for near-frontal views: A3 up to an obliqueness of about 1.03 and A4 up to about 1.1. The development set agrees (A4 reads every cell up to 1.07 and falls to 0.51–0.78 above 1.29). This is what the weak-perspective camera model predicts. Perspective projection divides by the depth Z; when the depth variation ΔZ across the object is small compared with its mean distance $\bar{Z}$, all points are divided by nearly the same number and the projection of a plane becomes affine. For a page of side s tilted by θ at distance $\bar{Z}$, $\Delta Z = s \sin\theta$, and the far side appears shorter than the near side by the factor $(\bar{Z} + \frac{1}{2}\Delta Z)/(\bar{Z} - \frac{1}{2}\Delta Z)$, which is the obliqueness measured here. A phone held about 30 cm above a 15 cm puzzle violates $\Delta Z \ll \bar{Z}$ as soon as the page is tilted noticeably, so a homography is required.

V. RECOGNITION

Cells and margin. The H4 image is divided into 81 equal 50×50 cells. Each cell has an inner margin of 7 px (14 %) on every side. The margin was chosen from measurements on the development set: printed digits lie between 11 and 40 px inside a cell, while grid lines occupy only the first few pixels next to the cell boundary, so 7 px separates the two.

Empty-cell criterion. The rectified image is binarised as in Stage 1 and split into connected components. A cell is filled when a component that is smaller than a cell and at least 30 % of the cell height has its centre inside the inner region of that cell. All other cells are empty and are never passed to the OCR engine. Grid lines are either larger than a cell or centred in the margin. Using the component instead of a hard crop keeps a digit complete when the rectification is a few pixels off; with a hard crop such digits touched the crop border and were lost. The empty-cell F1 score is 1.000 on the development set and 0.995 on the personal dataset. All five filled cells reported as empty are in the photograph of the curled page: three are rejected by this criterion and two are found but not read by Tesseract.

OCR configuration. Tesseract 5 is used through `pytesseract` with `--psm 10` (single character) and the whitelist `123456789`. Table V compares the page segmentation modes. Modes 10 and 7 are best on both datasets; modes 6, 8 and 13 are worse on isolated digits, and

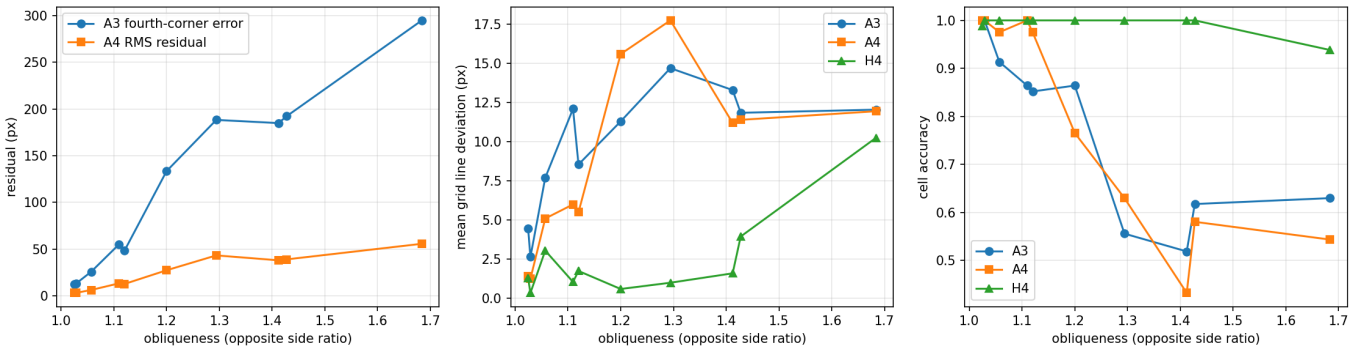


Fig. 2. Corner residuals, grid line deviation and cell accuracy against obliqueness (personal dataset).

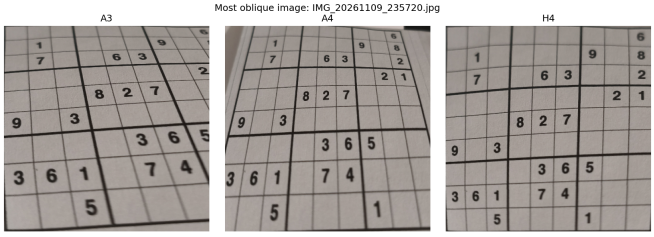


Fig. 3. A3, A4 and H4 rectifications of the most oblique personal photograph (obliqueness 1.68).

the whitelist makes no measurable difference once the digit is isolated.

Cell preparation. The digit mask is drawn in black on a white square whose side is 2.5 times the digit height, its strokes are thinned by one pixel, and the picture is resized to 40×40 px. This removes line remnants, centres the digit and gives Tesseract the white border it needs. Feeding the raw grey cell instead lowers cell accuracy from 0.996 to 0.943 on the development set and from 0.983 to 0.923 on my photographs. Two details mattered. Larger pictures were worse: at 96×96 Tesseract returned an empty string for about one digit in ten. Thinner strokes were better, because the blur before thresholding makes the mask slightly bold. Even with preparation, mode 10 occasionally returns nothing for a clear digit but rarely returns a wrong one. Since the cell is already known to be filled, an empty answer is retried with a padding of 2.0, 3.0 and 1.6 and finally with mode 13; this raises accuracy to 1.000 and 0.993.

Confusions. The final configuration makes no error on the development set and six errors in the 810 cells of the personal dataset. Five are filled cells reported as empty, all in the photograph of the curled page. Three of them (1, 2 and 7) are displaced by the curl until their centre lies on a grid line, so the empty-cell criterion rejects them. The other two (5 and 6) are found, but Tesseract returns nothing in all five attempts. The only true confusion is a sans-serif 1 read as 4 (Fig. 5). Tesseract answers 4 for this digit at a padding of 2.5 and 1 at the three other paddings; the pipeline keeps the first non-empty answer, so a majority vote over the paddings would have corrected it.

TABLE V
OCR VARIANTS (CELL ACCURACY)

Variant	Dev.	Personal
Prepared, psm 10 with retries (final)	1.000	0.993
Prepared, psm 10	0.996	0.983
Prepared, psm 10, no whitelist	0.996	0.984
Prepared, psm 7	0.996	0.983
Prepared, psm 6	0.970	0.931
Prepared, psm 13	0.935	0.900
Prepared, psm 8	0.934	0.893
Raw cells, psm 10	0.943	0.923

TABLE VI
RESULTS BY CONDITION

Group	n	Corner (%)	Cells	Empty F1	Exact
Development set					
All	20	0.28	1.000	1.000	1.00
C1	5	0.27	1.000	1.000	1.00
C2	9	0.27	1.000	1.000	1.00
C3	4	0.31	1.000	1.000	1.00
C4	6	0.27	1.000	1.000	1.00
C5	6	0.31	1.000	1.000	1.00
Personal dataset					
All	10	0.27	0.993	0.995	0.80
C1	2	0.16	1.000	1.000	1.00
C2	5	0.29	0.988	0.990	0.80
C3	2	0.19	0.994	1.000	0.50
C4	4	0.25	0.997	1.000	0.75
C5	3	0.32	1.000	1.000	1.00

VI. RESULTS

Table VI gives the metrics of Section 7 of the assignment, computed with `tools/evaluate.py`.

Robustness is 1.00 on both sets: every image was processed without a crash, in 2.7 to 4.3 s on a CPU. The corner error is the same on real and synthetic images. The largest value on my photographs is 0.52 %, on a photograph with a pen lying next to the grid. No error can be attributed to clutter or shadow: the errors come from the curled page (a C2 image) and from one OCR confusion (in an image labelled C3 and C4).

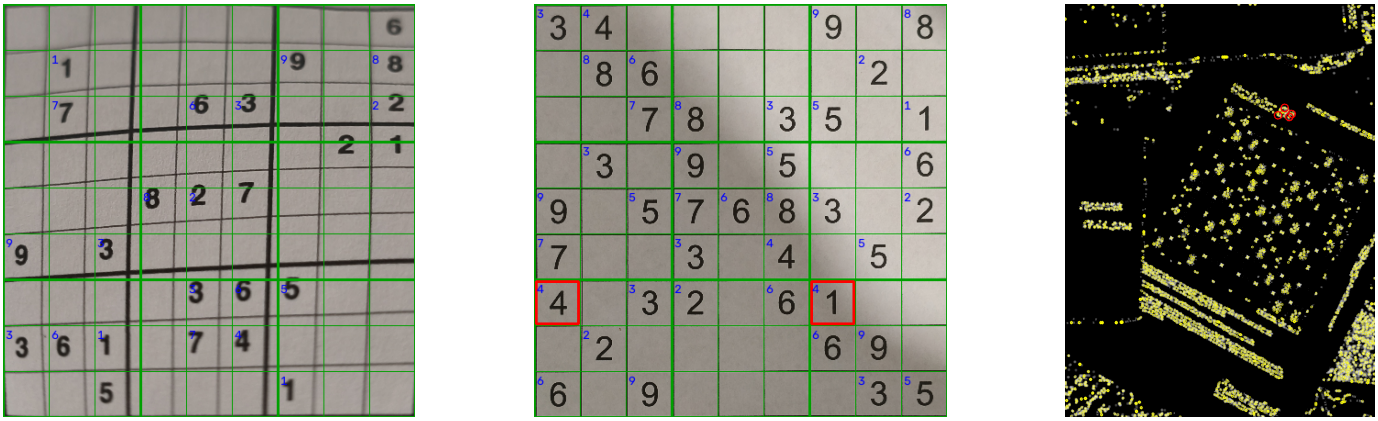


Fig. 4. Failure cases. Left: curled page after H4, with printed lines far from the ideal green grid. Centre: a 1 read as 4, with the conflicting cells marked in red. Right: Harris output for a puzzle cut by the image border; the selected corners (red) lie on the title text.

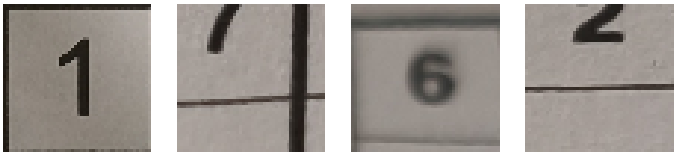


Fig. 5. Misread cells. From left: 1 read as 4; then 7, 6 and 2 reported as empty on the curled page.

VII. Failure Analysis

Six of the 810 cells of the personal dataset are wrong, five of them in one photograph. Fig. 4 shows the cases.

1. Curled page (C2, Fig. 4 left). The corners are correct (0.41 %), but the booklet page is curved, so the grid lines deviate by 10.3 px after H4 and whole rows are shifted by up to half a cell. Three digits end up centred on a grid line and are rejected as empty, and two more are located but not read (cell accuracy 0.938). The failing assumption is planarity in Stage 3; more than four points and a non-planar model would be needed.

2. Digit confusion (C3, Fig. 4 centre). Geometry is correct (0.20 %), but one 1 is read as 4. The validity check catches it: the grid then contains two 4s in one row, the verdict is *invalid* and both cells are marked. The failing stage is recognition.

3. Affine rectification of oblique views (Fig. 3). With A3 or A4 the grid lines converge, digits fall into the wrong cells, and cell accuracy is 0.43–0.63 for the four most oblique photographs, although the corners and the OCR stage are the same as for H4. The failing stage is the choice of model in Stage 3.

4. Puzzle cut by the image border (Fig. 4 right). In the first version of the rotated photograph one corner of the grid was outside the frame. The grid component touched the image border and was discarded by the region rule, and the component with the next best hole count was a piece of the title. The failing stage is region selection in Stage 2. The photograph was retaken with all four corners in the frame.

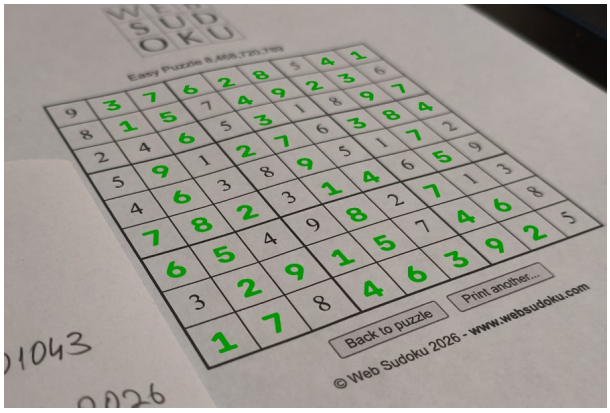


Fig. 6. B2: the solution (green) drawn on an oblique photograph.

VIII. Bonus Parts

The four bonus parts are functions in `sudoku_scan.py`. Their results, measured with `tools/bonus_study.py`, are given in Table VII.

B1 Solver (`solve`). The solver uses backtracking. The empty cell with the fewest candidates is filled first, its candidates are tried in ascending order, and the search goes back when a cell has no candidate left. The search does not stop at the first solution: a second solution proves that the solution is not unique. The answer is *unique*, *not unique*, *none* or *timeout* (5 s), and the time is measured with `time.perf_counter`. Every correctly read grid has a unique solution that equals the solution of the annotated grid, found in 40 ms at most. The two personal grids with OCR errors are detected: the curled page with five missing digits is *not unique*, and the grid with the misread 1 has *none*.

B2 Solution on the photograph (`draw_solution`). The missing digits are written at the cell centres of a blank 450×450 canvas. The canvas is warped to the size of the photograph with H_4^{-1} (`np.linalg.inv`, `cv2.warpPerspective`) and used as a mask to paint the digits on the photograph (Fig. 6). It is written only when the solution is unique.

B3 Many-point homography. The function `multipoint_homography` moves the Harris candidates to the rectified square with H4. A candidate within 8 px of a multiple of 50 in x and y belongs to that grid intersection. Harris gives up to four candidates around one line crossing, so they are averaged and refined with `cv2.cornerSubPix`. The 64 inner intersections and the four detected corners, at most 68 point pairs, are given to `cv2.findHomography` with RANSAC (threshold 3 px). The intersections on the outer border are not used: their candidates lie on the inner edge of the thick line, and with them the line deviation was worse than with H4. The result, HN, lowers the mean line deviation from 2.49 to 1.14 px on my photographs. The gain is largest on the three booklet pages that do not lie flat (3.1, 4.0 and 10.3 px with H4; 0.9, 1.0 and 3.5 px with HN), because 68 points spread over the grid give a better average plane than four corners. Cell accuracy does not change, so the grid is still read from H4. On the curled page only 10 of 20 point pairs are inliers, which again shows that this page is not a plane.

B4 Any orientation (`find_orientation`). The H4 image is turned by 0°, 90°, 180° and 270°. For each turn the digits found by the empty-cell criterion are placed on one small page, which is read with one `pytesseract.image_to_data` call, and the turn with the highest mean confidence is chosen. The corner list is then rolled so that it starts at the top-left corner of the puzzle. One call per turn costs about 0.4 s per image, much less than four full recognitions. The confidence of the upright turn is 38–96, and that of every other turn is at most 24. For the test every photograph was also turned by 90°, 180° and 270° and processed again: the orientation was found in all 120 runs. A turned photograph can differ from the upright one in one to three cells, because the detected corners move by up to one pixel in the rectified image.

TABLE VII
BONUS RESULTS

Measurement	Dev.	Personal
B1 unique solution found	20/20	8/10
B1 equal to true solution	20/20	8/8
B1 time, mean / max (ms)	10 / 39	6 / 24
B3 line dev. H4 (px)	1.01	2.49
B3 line dev. HN (px)	0.80	1.14
B3 cell acc. H4 / HN	1.000 / 1.000	0.993 / 0.991
B3 point pairs / inliers	66 / 65	62 / 56
B4 orientation found	80/80	40/40
B4 cell acc. at 0°	1.000	0.993
B4 cell acc. at 90°	0.999	0.990
B4 cell acc. at 180°	1.000	0.993
B4 cell acc. at 270°	1.000	0.986

IX. Conclusion and Limitations

The pipeline finds the puzzle from the structure of the grid itself (one component with 81 holes) and takes the corners from the Harris detector, with a corner error below 0.6 % on every real photograph. A homography is necessary for anything but near-frontal views; the affine models degrade in proportion to the ratio of opposite side lengths, as the weak-perspective model predicts.

Limitations: the region rule needs closed cells, a grid that does not touch other ink and a puzzle that is completely inside the frame; a single homography cannot correct a curled page; a digit that touches a grid line is merged with it and lost; and Tesseract in single-character mode still confuses some sans-serif digits. Ten photographs are a small sample, so the limits given for the affine models are indicative only.

APPENDIX
AI-USE DISCLOSURE

Tool	Purpose / stage	What you asked (summary)	What you kept, changed or rejected
Microsoft Copilot	Concept learning, writing	Harris, homography, OCR, report review	Used explanations only; implementation, Stage 2, and Stage 4 completed independently.
Claude	Coding support, debugging	OCR pipeline, preprocessing, OpenCV usage	Adapted suggestions; final code written and modified by me. Stage 2 and Stage 4 done independently.

I confirm that I wrote the corner-selection logic and the comparison analysis myself, and that I can explain and modify every part of my submission.

Name: Kerem Ataç Student no.: 042201043 Signature:  Date: 10.11.2026